SPACELONX ($SLX)

Smart Contract Audit Report

Version 1.0 - Preliminary professional security review

Date: 02 June 2026

Important scope limitation

This report is a professional preliminary audit document prepared from the project information available during review: the public contract repository README, declared token parameters, the Polygon Amoy contract address, and project architecture information. It is not a final mainnet certification because the complete Solidity source tree, test suite, deployment scripts and static-analysis outputs were not available inside the repository during this preparation session.

Project Snapshot

Field	Value
Token name / symbol	SpacelonX / SLX
Network reviewed	Polygon Amoy testnet
Contract address	0xDBF17Cc980Ae4f7db5119e8C9c3E2AD53023aC9E
Solidity / compiler	^0.8.20, compiled with 0.8.34
Libraries declared	OpenZeppelin Contracts v5.x
Token supply	10,000,000,000 SLX, fixed supply, 18 decimals
Contract repository	https://github.com/silversurfer239/spacelonx-contract
Explorer reference	https://amoy.polygonscan.com/address/0xDBF17Cc980Ae4f7db5119e8C9c3E2AD53023aC9E

Executive Summary

SpacelonX (SLX) is documented as an ERC-20 community / parody token using OpenZeppelin v5 components, with a fixed supply of 10,000,000,000 SLX, burn functionality, configurable buy/sell taxes, and vesting allocations. The current deployment reviewed is on Polygon Amoy testnet, not mainnet.

From a security perspective, the documented architecture is consistent with a conventional ERC-20 memecoin pattern: fixed supply, no post-deploy minting, capped tax parameters and vesting schedules. The highest risk area is not an identified exploitable bug in the reviewed material; it is the absence of full reproducible source-code and test evidence in the public GitHub repository during this preparation phase.

The project should not be launched on mainnet without a complete source-level audit, unit/invariant tests, deployment reproducibility, exact constructor parameter verification, multisig ownership controls and a public incident-response plan.

Findings Summary

Severity	Count
Critical	0
High	1
Medium	3
Low	1
Informational	1

ID	Severity	Status	Finding
SLX-AUD-01	High	Open	Final source-code audit and automated security tooling not yet evidenced in repository
SLX-AUD-02	Medium	Review Required	Owner-controlled tax configuration requires governance guardrails
SLX-AUD-03	Medium	Review Required	Revocable vesting allocations should be transparent and event-driven
SLX-AUD-04	Medium	Open	No public evidence of test coverage, invariant testing or deployment reproducibility
SLX-AUD-05	Low	Open	Project links should be updated from Netlify to GitHub Pages
SLX-AUD-06	Informational	Acknowledged	Testnet-only deployment status must remain explicit

Scope and Methodology

Scope included a documentation-based architectural review of the SLX token model, tokenomics, tax/vesting design and deployment posture.

The review checked the public repository metadata, documented supply model, vesting schedule, declared OpenZeppelin dependencies, the Polygon Amoy contract address and the public project communication stack.

This report uses a practical severity model: Critical, High, Medium, Low and Informational. Severity combines likelihood, impact and exploitability. Because complete Solidity source was not available in the reviewed repository, several items are marked as design or process risks rather than confirmed code-level vulnerabilities.

What was not performed

No Slither, Mythril, Foundry invariant tests, Echidna fuzzing, bytecode equivalence proof, storage-layout diff, or constructor-argument verification was performed in this document generation step. These are recommended before any production deployment.

Documented Contract Properties Reviewed

Control	Review Status	Observation
ERC-20 fixed supply	Partially verified from documentation	10B SLX fixed supply documented.
No mint after deploy	Documented	README states no mint after deploy.
Burn functionality	Documented	ERC20Burnable is declared by README.
Tax cap enforcement	Documented, source review pending	10% immutable cap documented; implementation requires source verification.
Vesting cliffs and release math	Design reviewed, source review pending	Schedule and durations documented; release logic requires source-level inspection.
Owner privileges	Requires source review	Ownable is documented; exact privileged functions should be verified.
DEX/pair configuration	Requires source review	Tax routing usually depends on pair detection; implementation not reviewed here.
Reentrancy exposure	Likely low for pure ERC-20 transfers, source review pending	No ETH withdrawal logic evidenced, but final code must be checked.
Overflow/underflow	Likely mitigated by Solidity 0.8+	Solidity 0.8+ has checked arithmetic by default.
Dependency risk	Acceptable if OpenZeppelin v5.x imported unmodified	Use exact pinned version for reproducible deployment.

Detailed Findings

SLX-AUD-01 - Final source-code audit and automated security tooling not yet evidenced in repository

Severity: High | Status: Open

Description: The public GitHub contract repository observed during this review provides project-level metadata and states that the source is verified on Polygon Amoy. However, the Solidity source files and test suite were not available in the repository during this document preparation. This limits reproducibility and prevents a full source-level review from the repository alone.

Impact: Mainnet launch with unreviewed source or missing test evidence could expose the project to logic errors, privileged-role mistakes or tokenomics misconfiguration.

Recommendation: Before mainnet deployment, publish the complete Solidity source tree, deployment scripts, constructor arguments, test suite, coverage report, Slither/Mythril outputs and exact compiler settings. Run a full line-by-line audit after source publication.

SLX-AUD-02 - Owner-controlled tax configuration requires governance guardrails

Severity: Medium | Status: Review Required

Description: The README states buy tax = 2%, sell tax = 3%, and max tax hard cap = 10%. If these values are owner-controlled, this creates a centralization and trust surface. The cap reduces the risk but does not remove governance concerns.

Impact: The token design includes configurable buy and sell taxes, capped at 10%. Even with a hard cap, users should understand who can change tax parameters and under what governance process.

Recommendation: Use a multisig for owner authority, publish a tax-change policy, add time delay / timelock for sensitive tax updates where possible, and emit explicit events on all configuration changes.

SLX-AUD-03 - Revocable vesting allocations should be transparent and event-driven

Severity: Medium | Status: Review Required

Description: The project documentation indicates 40% of supply is locked across vesting schedules, with advisor and ecosystem allocations marked revocable. This should be clear in the contract events and public documentation.

Impact: Revocable advisor/ecosystem vesting can be legitimate, but unclear revocation powers may create community trust risk and operational ambiguity.

Recommendation: Document the exact revocation authority, conditions and destination of revoked tokens. Ensure VestingRevoked, TokensReleased and BeneficiaryUpdated events exist where applicable.

SLX-AUD-04 - No public evidence of test coverage, invariant testing or deployment reproducibility

Severity: Medium | Status: Open

Description: No test files were identified in the contract repository during this preparation phase. The absence of tests does not prove they do not exist, but it prevents external reviewers from validating behavior.

Impact: Token contracts with taxation and vesting logic depend on correct edge-case handling: transfers, exemptions, pair configuration, vesting cliffs, revoked allocations and owner operations.

Recommendation: Publish a test suite covering transfers, burns, vesting releases, cliff behavior, revocation behavior, tax cap enforcement, zero-address checks, ownership transfer and DEX pair configuration.

SLX-AUD-05 - Project links should be updated from Netlify to GitHub Pages

Severity: Low | Status: Open

Description: The contract README references the former Netlify website. The operational site now appears to be the GitHub Pages deployment used by the SpacelonX Journal.

Impact: Outdated links can mislead users and auditors, especially if the primary website has migrated away from Netlify.

Recommendation: Update repository documentation, website footer and official communication channels to consistently point to the current GitHub Pages URL until a custom domain is configured.

SLX-AUD-06 - Testnet-only deployment status must remain explicit

Severity: Informational | Status: Acknowledged

Description: The project currently documents deployment on Polygon Amoy testnet. This is appropriate for testing, but every public page should clearly distinguish testnet from any future mainnet contract.

Impact: Users may incorrectly assume that the token is already deployed on mainnet if testnet status is not clearly communicated.

Recommendation: Keep a single canonical contract-address page. When mainnet is deployed, archive the Amoy address as testnet and display the mainnet address prominently.

Security Review by Risk Domain

Supply and minting: The fixed-supply model and absence of post-deploy minting are positive controls if implemented exactly as documented. Verify constructor mints only the intended supply and no mint function remains callable after deployment.

Tax logic: Tax logic is one of the highest complexity areas in memecoin contracts. Review pair detection, tax exemptions, wallet routing, zero-tax edge cases, maximum tax cap and event emission. Confirm tax cannot exceed the documented cap.

Ownership and privileged roles: Ownable administration must be minimized. A single externally-owned owner key is not recommended for mainnet. Use a multisig and publish operational procedures.

Vesting: Vesting logic must be carefully tested around cliffs, release rounding, revocation, total vested amount and beneficiary accounting. Tests must cover boundary timestamps.

Reentrancy: Standard ERC-20 transfers are generally not reentrancy-prone unless external calls, fee routers, DEX interactions or callbacks are introduced. Final source must be reviewed for any external call path.

Arithmetic: Solidity 0.8+ provides checked arithmetic by default, but percentage calculations, rounding and total-accounting invariants must still be tested.

DEX/mainnet launch: Before any liquidity pool, verify pair configuration, liquidity lock, tax exclusions, ownership transfer and emergency response. Never reuse testnet addresses as mainnet references.

Recommended Pre-Mainnet Checklist

- Publish the full Solidity source tree in the official GitHub contract repository.
- Pin exact OpenZeppelin dependency versions and compiler version in package lock files.
- Publish deployment scripts and exact constructor arguments.
- Run and publish unit-test results and coverage report.
- Run Slither static analysis and remediate all High/Medium issues.
- Run Mythril/Manticore-style symbolic analysis where practical.
- Run Foundry or Hardhat invariant tests for supply, tax cap and vesting accounting.
- Deploy to a fresh testnet, verify source, and compare bytecode against build artifacts.
- Use a multisig owner before mainnet deployment.
- Define tax-change policy and publish governance transparency rules.
- Lock liquidity before any public mainnet launch announcement.
- Create a canonical contract-address page and warn users against impostor contracts.

Mainnet Readiness Assessment

Area	Readiness	Notes
Contract architecture	Moderate	ERC-20 + OpenZeppelin foundation is conventional, but final code review is still required.

Documentation	Moderate	Token parameters and vesting are documented. Official links should be updated.
Security evidence	Low to Moderate	Repository lacks visible tests/source artifacts in this review context.
Operational controls	Pending	Multisig, tax-change policy and liquidity lock evidence should be produced.
Community transparency	Moderate	Website, Journal, X and Telegram are operational; contract address clarity must be maintained.

Conclusion

The documented SLX design does not reveal an obvious fatal architectural flaw at the documentation-review level. Fixed supply, no mint after deploy, OpenZeppelin v5 usage, tax caps and vesting schedules are positive design signals.

However, this report should be treated as a preliminary audit report, not a final security certification. The main unresolved issue is evidence: a complete source tree, tests, deployment reproducibility and static-analysis outputs must be published before a mainnet launch.

The most important next step is to perform a full source-code audit on the verified Solidity contract, reconcile it with the GitHub repository, test tax and vesting edge cases, and move administrative control to a multisig before any production deployment.

References

- SpacelonX contract repository: https://github.com/silversurfer239/spacelonx-contract
- Polygon Amoy verified contract page: https://amoy.polygonscan.com/address/0xDBF17Cc980Ae4f7db5119e8C9c3E2AD53023aC9E
- OpenZeppelin Contracts documentation: https://docs.openzeppelin.com/contracts/
- Solidity Security Considerations: https://docs.soliditylang.org/en/latest/security-considerations.html
- OWASP Smart Contract Top 10: https://owasp.org/www-project-smart-contract-top-10/

Audit Disclaimer

This document is not financial advice, legal advice, investment advice or a guarantee of security. Smart contract risk cannot be fully eliminated. The absence of a finding in this report does not mean the absence of vulnerabilities. A final production audit requires full source-code review, test execution, bytecode verification and mainnet deployment review.